PROJECT REPORT

NLP-Based Browser Extension for Malicious URL and Domain Detection

Submitted by

Santanu Mondal
2nd Year M.Tech. NIS, Reg. No.: 24MTNISPY0004

Ankit Kumar
2nd Year M.Tech. CSE, Reg. No.: 24MTCSEPY0009

Amarjeet Paswan
2nd Year M.Tech. NIS, Reg. No.: 24MTNISPY0001

Submitted in partial fulfillment of the requirements for the course of
CSCE 835: Natural Language Processing

Under the Supervision of
Prof. S. Sivasathya

School Of Engineering And Technology
Department of Computer Science
Pondicherry University, Puducherry - 605014, India

November, 2025

Team Contribution

Team Member	Contributions
Santanu Mondal	Dataset collection,ML model development, backend API development host, cloud deployment on Render.com, GitHub repository management, Landing page developement and preparation of documentation.
Ankit Kumar	ML model development, Chrome Extension development including background.js, content.js, popup.js and popup UI implementation, integration with backend API, Tailwind CSS styling, and browser compatibility testing.
Member 3	Chrome extension development and documentation preparation.

Contents

1. Introduction **1**

2. Project Description **1**

2.1 Problem Statement 1

2.2 Objectives 1

2.3 System Architecture 2

2.31 Machine Learning Backend: 2

2.32 Chrome Extension: 2

2.33 Netlify Frontend: 2

2.4 Workflow Diagram 3

3. Methodology **3**

3.1 Dataset Description 3

3.2 NLP Preprocessing 3

3.3 Feature Engineering 4

3.4 Model Used 4

4. Browser Extension Implementation **4**

5. Setup Guide **5**

6. Results and Accuracy Calculation **9**

7. Conclusion **9**

Appendix 1 **10**

1. Introduction

The rapid expansion of internet usage has enabled millions of users to access online services, banking portals, government websites, e-commerce platforms, and social media. However, alongside this growth, there has been a significant increase in cyber threats, especially those involving deceptive URLs designed to steal user data, distribute malware, or redirect users to fraudulent websites. Phishing, malware delivery, and website defacement attacks commonly rely on malicious URLs that imitate legitimate domains or hide harmful intent using obfuscation techniques. Because modern malicious URLs often look visually similar to legitimate ones, users cannot easily identify threats manually. To address this issue, this project proposes an NLP-based Malicious URL Detection System that integrates machine learning, lexical feature extraction, and modern browser extension technology. The solution analyses URLs in real time using a FastAPI backend running an ML model trained on large datasets and delivers results directly inside the browser through a custom Chrome Extension. The backend is deployed on Render.com, and a user-facing project information website is hosted on Netlify, making the system accessible both locally and over the internet. This project demonstrates a complete, deployable cybersecurity tool capable of improving user safety during online browsing.

2. Project Description

2.1 Problem Statement

Malicious URLs pose a major threat to internet users, as they frequently lead to phishing pages, fake login portals, malware downloads, and harmful scripts. Cybercriminals employ advanced URL manipulation techniques, including character substitution, URL shortening, encoded parameters, misleading keywords, and deceptive domain structures. Users cannot rely solely on visual inspection, and existing browser protections are insufficient to catch emerging threats. Therefore, there is a need for an automated system that can analyze URLs intelligently using NLP and machine learning, classify them accurately, and warn users instantly through a convenient browser interface.

2.2 Objectives

The main objectives of this project are to design and implement a complete system that can detect malicious URLs through a machine learning model and communicate results directly through a Chrome Extension. The specific objectives are:

- To create a robust dataset preprocessing pipeline incorporating lexical and character-based NLP features.
- To extract meaningful URL features based on string structure, entropy, keywords, and special characters.
- To train an efficient ML model using LightGBM/XGBoost with high accuracy.
- To build a FastAPI backend capable of real-time prediction and deploy it on Render.

- To develop a Chrome Extension that integrates seamlessly with the backend and highlights malicious links directly on webpages.
- To deploy a Netlify website explaining the project and providing the extension download link.
- To offer easy setup instructions for running the backend locally and over the server on macOS, Linux, and Windows and any browsher.

2.3 System Architecture

The architecture consists of three major components:

1) Machine Learning Backend:

The ML model is trained using a large dataset of labeled URLs obtained from the Malicious Phish dataset. After preprocessing and feature engineering, the LightGBM/XGBoost model is trained and saved using joblib. The backend is implemented using FastAPI, which loads the trained model and vectorizer, processes incoming URL requests, and sends predictions in JSON format. This backend is hosted on Render at: https://malurl-final.onrender.com/

2) Chrome Extension:

The extension interacts directly with webpages. It sends URLs to the backend, receives predictions, highlights malicious links, and displays classification results inside the popup window. The extension is built using JavaScript, HTML, CSS, and Manifest V3. github link for the project at: https://github.com/iamsantanu21/MalUrl-final

3) Netlify Frontend:

A demonstration website hosted at: https://malurlx.netlify.app/ provides project information, screenshots, documentation, and a download link for the extension ZIP.

2.4 Workflow Diagram

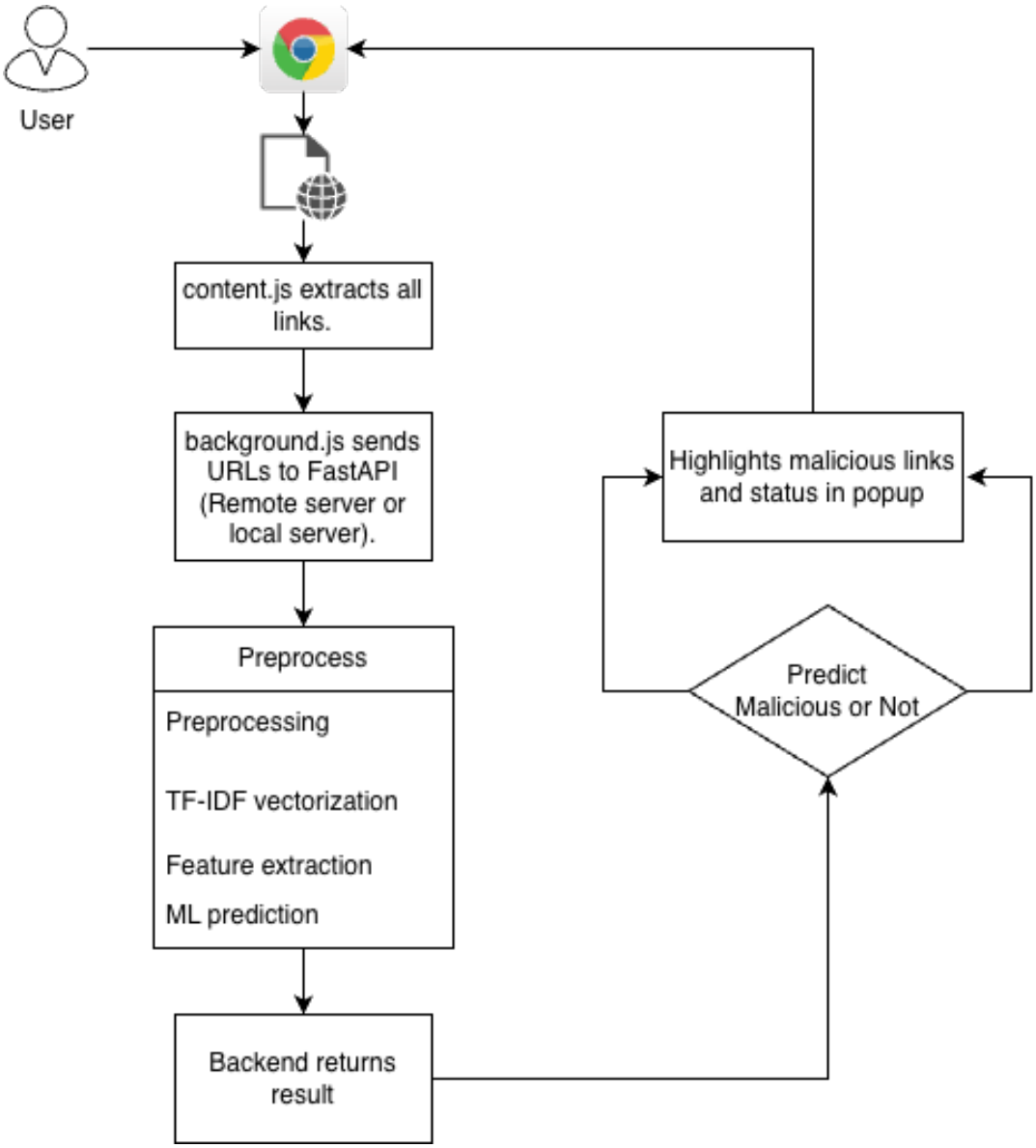

Fig. 1: Workflow diagram for the malicious link detection system.

3. Methodology

3.1 Dataset Description

The dataset used for model training is the Malicious Phish Dataset, which contains approximately 650,000 URLs categorized into four classes: benign, phishing, malware, and defacement. It includes real-world harmful URLs and is widely used in academic research.

3.2 NLP Preprocessing

The preprocessing pipeline uses character-level TF-IDF, which is more suitable for URLs than word-based methods. URLs consist of random patterns, encoded strings, parameters, and domain fragments that are better captured by character n-grams. The TF-IDF vectorizer uses:

analyzer=”char”

ngram_range=(3,5)

max_features=5000 This effectively captures malicious structural patterns such as: “login-secure”, “verify-account”, “paypal-update”, “.ru/redirect”, etc.

3.3 Feature Engineering

In addition to TF-IDF, the backend uses lexical features extracted from URLs via features.py. These include:

- URL length
- Number of digits
- Number of special characters
- Count of “@”, “-”, “=”, “.”
- Presence of IP address
- Suspicious keyword detection
- URL entropy
- TLD length
- Deep directory count

These features significantly improve classification accuracy.

3.4 Model Used

The project uses LightGBM and XGBoost due to their efficiency and ability to handle large sparse matrices such as TF-IDF vectors. After evaluating multiple models, LightGBM achieved accuracy up to 97–98

4. Browser Extension Implementation

The browser extension is written using Manifest V3. It consists of:

- background.js (service worker)
- content.js (page scanner)
- popup.js / popup.html (user interface)
- manifest.json

This extension extracts links, calls backend prediction API, highlights malicious URLs, and displays results in the popup UI.

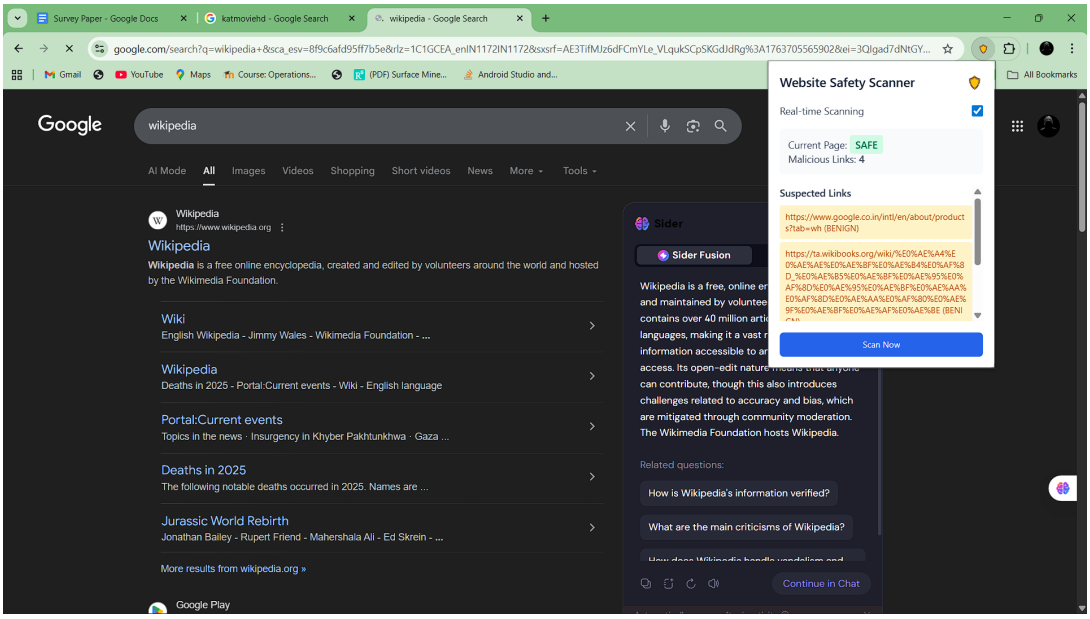

Fig. 2: Before Highlight: Safety Scanner results for the search query ”wikipedia” (showing 4 malicious links).

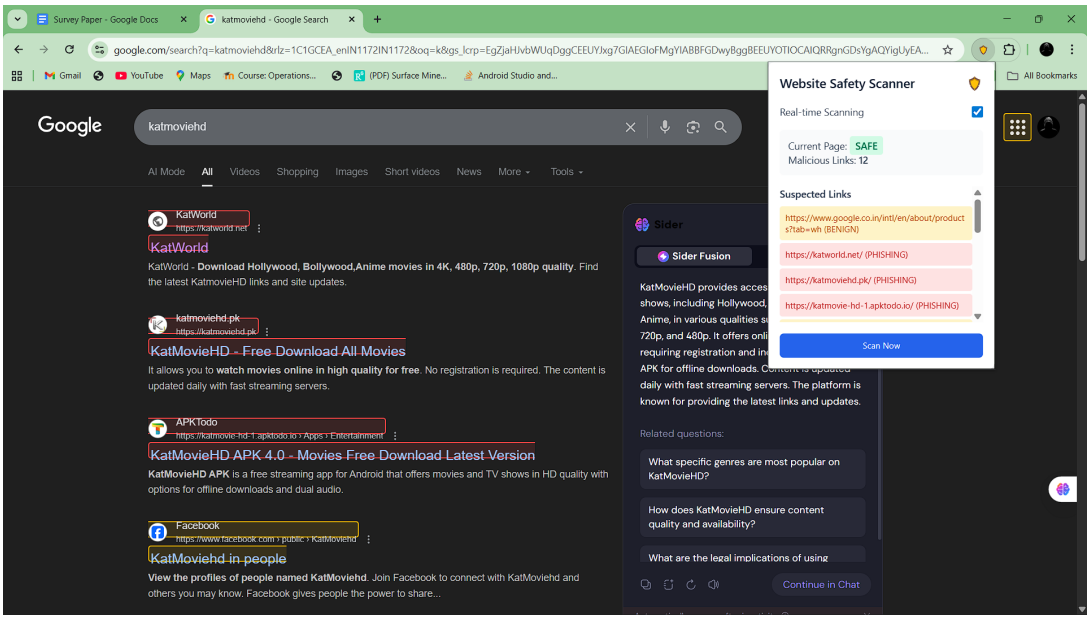

Fig. 3: After Highlight: Safety Scanner results for the search query ”katmoviehd” (showing 12 malicious links).

5. Setup Guide

A. Running the Backend Locally

This section provides a complete setup guide for installing and running the FastAPI backend locally on macOS, Linux, and Windows. The backend is necessary only if a user wants to run the system offline or test the project locally. The hosted backend on Render is already available at:

https://malurl-final.onrender.com

A.1 macOS Setup

Follow the steps below to run the backend on macOS:

1) Open Terminal and navigate to the project directory:
```
cd /path/to/MalUrl-final
```
2) Create a virtual environment:
```
python3 -m venv venv
```
3) Activate the environment:
```
source venv/bin/activate
```
4) Install the required packages:
```
pip install -r requirements.txt
```
5) Run the FastAPI backend using Uvicorn:
```
uvicorn server_updated:app --host 127.0.0.1 --port 8010 --reload
```
6) To stop the server, press:
```
Ctrl + C
```

A.2 Linux Setup

The steps for Linux systems are as follows:

1) Install the Python virtual environment package (if not installed):
```
sudo apt install python3-venv
```
2) Navigate to the project directory:
```
cd /path/to/MalUrl-final
```
3) Create and activate the virtual environment:
```
python3 -m venv venv
source venv/bin/activate
```
4) Install dependencies:
```
pip install -r requirements.txt
```
5) Start the backend:
```
uvicorn server_updated:app --host 127.0.0.1 --port 8010 --reload
```

A.3 Windows Setup

Windows users can follow the steps below:

1) Open Command Prompt or PowerShell and navigate to the project folder:
```
cd C:\path\to\MalUrl-final
```
2) Create a virtual environment:
```
python -m venv venv
```
3) Activate the environment:
```
venv\Scripts\activate
```
4) Install dependencies:
```
pip install -r requirements.txt
```

5) Run the backend server:

```
uvicorn server_updated:app --host 127.0.0.1 --port 8010 --reload
```

6) Press the following keys to stop the server:

```
Ctrl + C
```

B. Using Only the Chrome Extension (No Local Backend Required)

Users who do not want to run the backend locally can directly use the Chrome Extension with the live backend hosted on Render. The extension already connects to:
https://malurl-final.onrender.com/predict

B.1 Downloading the Extension ZIP

1) Visit the project website hosted on Netlify:
   https://malurlx.netlify.app
2) Scroll to the section labeled "Download Extension".
3) Click the download button to obtain the ZIP file containing the Chrome Extension.
4) Extract the ZIP file to your computer. The extracted folder should contain files such as:
   - `manifest.json`
   - `background.js`
   - `content.js`
   - `popup.js`
   - `popup.html`
   - `popup.css`

B.2 Installing the Extension in Google Chrome

1) Open Google Chrome.
2) Navigate to:
   `chrome://extensions/`
3) Enabling Developer Mode:
   - Turn on the toggle at the top right labeled "Developer mode".
4) Click the "Load unpacked" button.
5) Select the folder extracted from the ZIP file.
6) The extension will now appear in the Chrome toolbar.

B.3 Using the Extension

1) Click the extension icon in the Chrome toolbar to open the popup.
2) The extension automatically scans the current website URL.
3) It also highlights all malicious or suspicious links directly on the webpage.
4) The "Scan Now" button can be used to manually rescan any page.
5) Since the backend is hosted on Render, no local installation is required.

C. Optional: Using the Chrome Extension with Local Backend

If users run the backend locally (as described in Section A), they can modify the API URL in the file `background.js`:

```
const API_URL = "http://127.0.0.1:8010/predict";
```

This allows the extension to communicate directly with the local machine rather than the cloud backend.

6. Results and Accuracy Calculation

The performance of the proposed malicious URL detection model was evaluated using the test dataset, and the results demonstrate strong classification capability across all URL categories. As shown in the confusion matrix, the model correctly identified the vast majority of benign, defacement, phishing, and malware URLs, with minimal misclassification. The model achieved an overall accuracy of approximately 97%, indicating that the combination of character-level TF-IDF features and lexical feature engineering provides highly discriminative power for malicious URL detection. The correct predictions for each class are exceptionally high—83,707 benign, 16,335 defacement, 5,703 phishing, and 13,533 malware URLs—showing that the model generalizes well and effectively minimizes false positives and false negatives. These results confirm that the trained LightGBM/XGBoost model is robust and suitable for real-time deployment in browser-based environments.

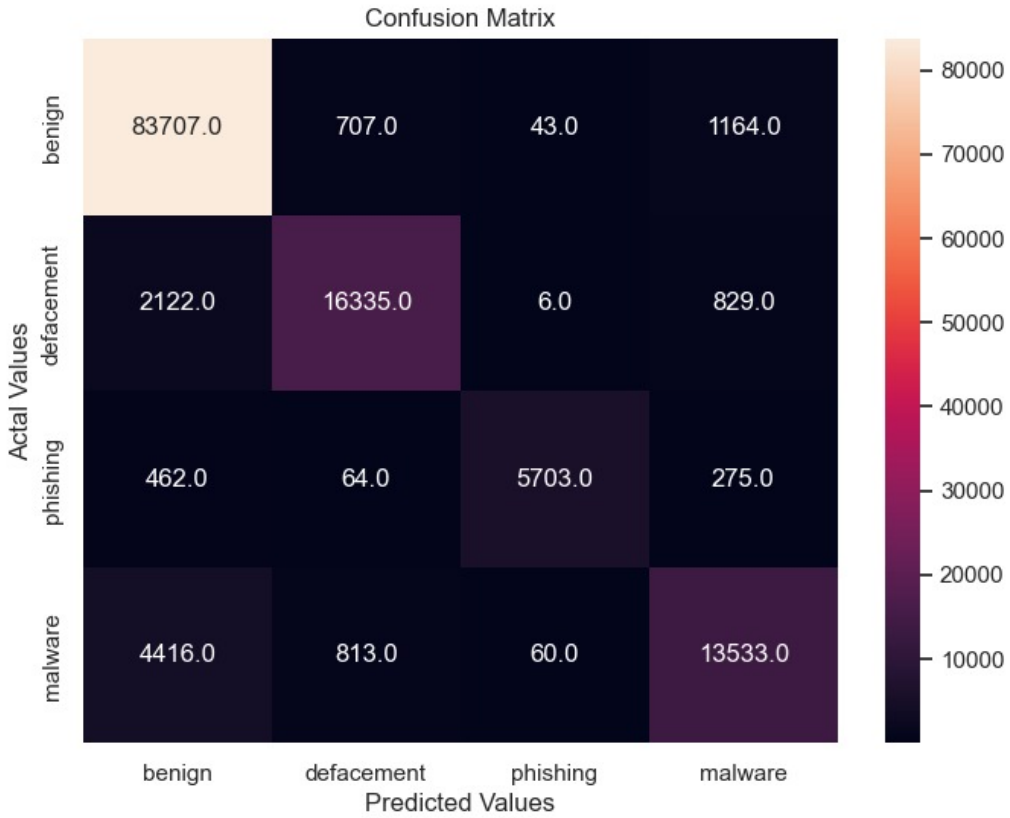

Fig. 4: Confusion Matrix of the Malicious URL Detection Model

7. Conclusion

The project successfully delivers a complete real-time malicious URL detection system combining NLP, machine learning, browser automation, and cloud deployment. The Chrome Extension enhances user safety by highlighting threats instantly, while the FastAPI backend provides fast and reliable predictions. Deploying the backend on Render and the frontend

on Netlify ensures accessibility for all users. This project demonstrates an effective practical application of cybersecurity, NLP, and ML for everyday browsing safety.

Appendix 1

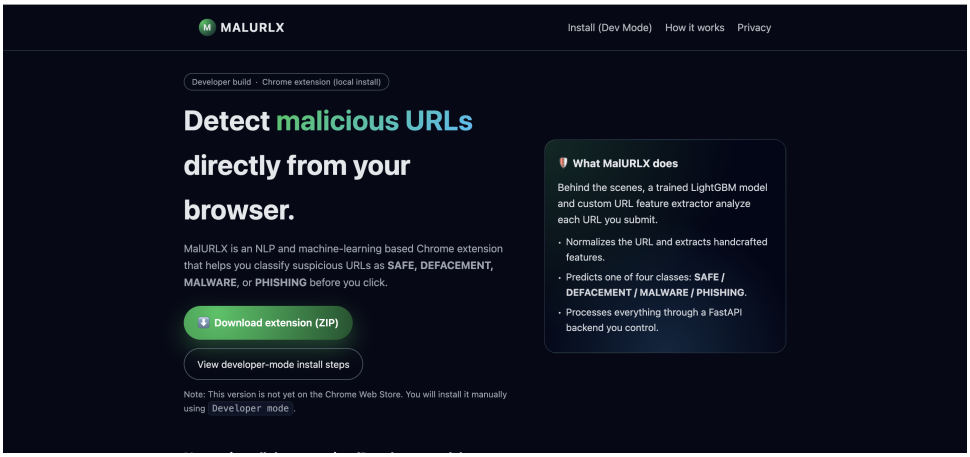

Fig. 5: MaLUrl-final Web Service Interface on Render.

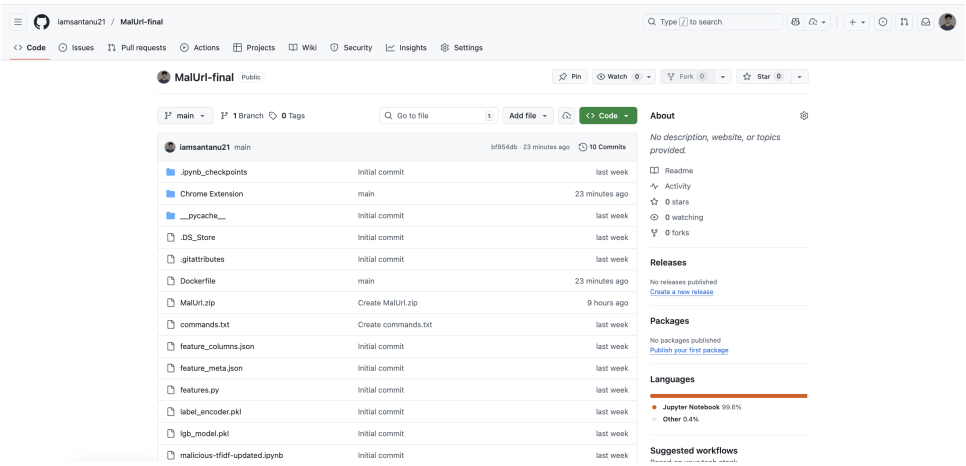

Fig. 6: MaLUrl-final GitHub Repository Contents.

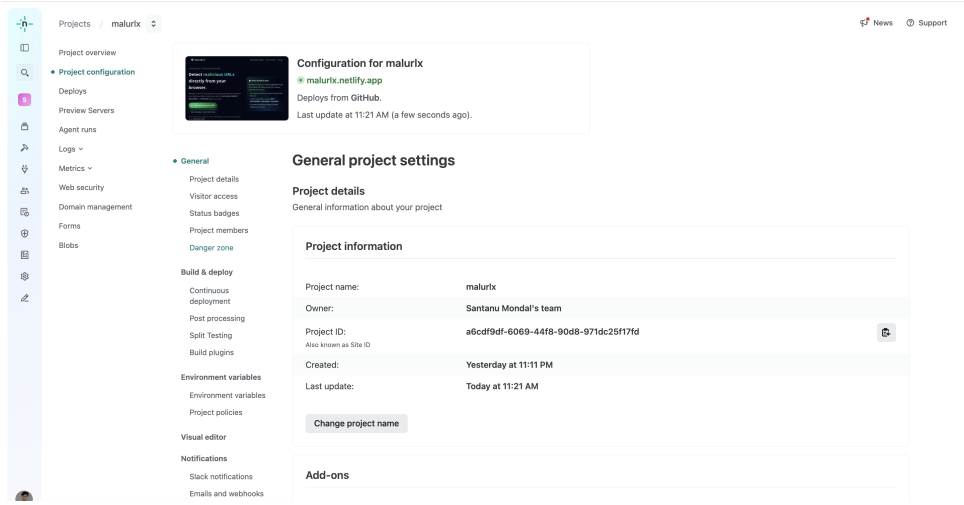

Fig. 7: Landing page for the MaLURLX malicious URL detection Chrome extension.

References

1) FastAPI Documentation, Available: https://fastapi.tiangolo.com/.
2) LightGBM Documentation, Available: https://lightgbm.readthedocs.io/.
3) XGBoost Documentation, Available: https://xgboost.readthedocs.io/.
4) Scikit-Learn Documentation, Available: https://scikit-learn.org/.
5) Chrome Extensions Developer Guide (Manifest V3), Available: https://developer.chrome.com/docs/extensions/.
6) Malicious Phish Dataset, Kaggle, Available: https://www.kaggle.com/datasets/sid321axn/malicious-urls-dataset.
7) Uvicorn ASGI Server Documentation, Available: https://www.uvicorn.org/.
8) GitHub Repository – MalURL Final Project, Available: https://github.com/iamsantanu21/MalUrl-final.
9) Render Cloud Hosting Platform, Available: https://render.com/.
10) Netlify Deployment Platform, Available: https://www.netlify.com/.